Given the matrix,

$$A = \begin{bmatrix} 2 & 3 \\ -1 & 4 \end{bmatrix}$$

its inverse is equal to:

$$A^{-1} = \begin{bmatrix} 4/12 & -3/12 \\ 1/12 & 2/12 \end{bmatrix}$$

And if we multiply $A*A^{-1}$, then the result is:

$$A*A^{-1} = \begin{bmatrix} 2 & 3 \\ -1 & 4 \end{bmatrix} * \begin{bmatrix} 4/12 & -3/12 \\ 1/12 & 2/12 \end{bmatrix} = \begin{bmatrix} (8/12 + 3/12) & (-6/12 + 6/12) \\ (-4/12 + 4/12) & (3/12 + 8/12) \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = I$$

Of course, we just as well could have multiplied $A^{-1}*A$, and the result also would have been the identity matrix I.

Concatenation

The final topic we are going to touch upon is one of the coolest things about matrices, and that's the property of *concatenation*. Concatenation means that a single matrix can be generated by multiplying a set of matrices together:

$$C = A_1*A_2*A_3*...*A_n$$

The interesting thing is that now the single matrix C holds all of the transformation information that the other matrices contained. This is a very useful tool when performing multiple transformations to all the points that make up the objects of the game. We can use the single concatenated matrix, which is the product of all of the desired transformations, instead of multiplying each point by two or three different matrices that each perform a single operation such as translation, scaling, and rotation.

Using this technique speeds up the game calculations by about 300 percent! So this is a good thing to know. However, as we learned earlier, the order of multiplication is very important. When we concatenate matrices, we must multiply them in the order in which we wish them to operate, so make sure that translation is performed first or rotation is done last (or whatever).

Fixed Point Mathematics

The last purely mathematical topic we must discuss is *fixed point mathematics*. If you know anything about 3D graphics, you're sure to know something about fixed point math. Fixed point math is a form of mathematics that is based on the use of integers to approximate decimals and fractions. Normally, we do calculations using floating point numbers. However, there is a problem with floating point numbers

and microprocessors–they don't get along. Floating point math is very complex and slow, mainly because of the format of a floating point number as it is represented in the computer.

Floating point numbers use a format called IEEE, which is based on representing any number by its *mantissa* and *exponent*. For example, the number 432324.23123 in floating point is represented like this,

4.3232423123E+5

which might be more familiar as:

4.3232423123×10^5

In either case, the IEEE format stores the mantissa and the exponent in a way that is a computational hazard to say the least. The results of this format are that multiplication and division are slow, but addition and subtraction are really slow! Doesn't that seem a bit backward? Fortunately, as time has marched on, most CPUs, including the 486DX and 586 Pentium, have on-chip math coprocessors. However, only the latest Pentium processor has floating point performance that exceeds integer performance (sometimes). Thus, we have to make a decision–whether or not to assume that everybody has a Pentium. That would be a mistake, since it will be two to three years before the majority of people have converted. Hence, we must assume that most users have a 486DX (even that is pushing it).

The floating point performance of a 486DX is good, but it's still not as good as integer performance (especially for addition and subtraction). Moreover, since graphics are performed on an integral pixel matrix, at some point, floating point numbers must be converted to integers, and this is a real performance hit. The moral to the story is that we are going to avoid math as much as possible, but we *are* going to use floating point numbers for the majority of our calculations during the prototyping phase of our graphics engine. Then when we are ready to make the engine really rock, we will convert all the floating point math to fixed point math to get the best possible speed.

Another reason to use floating point math during the prototype phase of our 3D graphics engine is that the graphics are hard enough to understand without a ton of confusing fixed point calculations getting in the way of the purity of everything. And many people might just want to keep the floating point library and forget the final fixed point version, so this gives everyone the best options.

So what exactly is fixed point math? Fixed point math is a form of computer math that uses an integer to represent both the whole part and fractional part of a decimal number by means of *scaling*. The main premise is that all numbers are represented in binary; therefore, if we use a large integer representation such as a LONG, which is 32 bits, we can play a little game. The game has the rules that the whole part of a number will be placed in the upper 16 bits and the decimal part will be placed in the lower 16 bits. Figure 10-35 shows this breakdown. Although

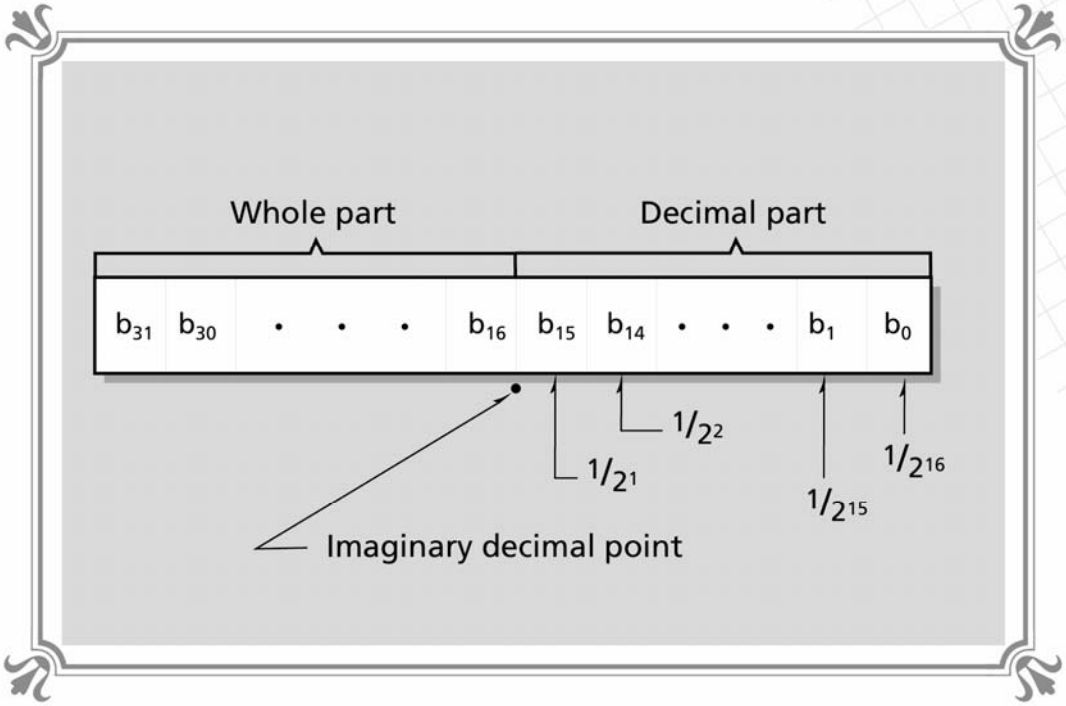

FIGURE 10-35
Typical fixed point representation

we can place the "fixed point" anywhere, most implementations place it smack in the middle for simplicity.

The lower 16 bits are called the *decimal part* and the upper 16 bits are called the *whole part*. All we need is to come up with a set of rules so that we can assign numbers to fixed point numbers. Well, it turns out to be quite easy. First, let's define a fixed point type:

```
typedef long fixedpoint;
```

We know that the whole part is supposed to be in the upper 16 bits and the decimal part in the lower 16 bits; therefore, if we want to convert the simple integer 10 into a fixed point number, we can do something like this:

```
fixedpoint f1;

f1 = (fixedpoint)(10 << 16);
```

What's happening is, we are scaling or shifting the 10 by 16 bits (which is equivalent to multiplying by 65536) to place it into the upper half of the fixed point number where it belongs. But how can we place a decimal number into a fixed point number? That's easy–instead of shifting the number, we must multiply it by 65536, like this:

```
f1 = (fixedpoint)(34.23432 * 65536);
```

The reason for this is that the compiler represents all decimal numbers as floating point numbers and shifting doesn't make sense, but multiplying does. Now that

we can assign a fixed point number with a standard integer or a decimal number, how do we perform addition, subtraction, multiplication, and division? Addition and subtraction are simple. Given two fixed point numbers f1 and f2, to add them we use the following code:

```
f3 =f1+f2;
```

And to subtract them we use:

```
f3 = f1-f2;
```

Isn't that easy! The problem occurs when we try to multiply or divide fixed point numbers. Let's begin with multiplication. When two fixed point numbers are multiplied, there is an overscaling because each fixed point number is really multiplied internally by 65536 (in our format); therefore, when we try to multiply two fixed point numbers, like this,

```
f3 = f1*f2;
```

the result is overscaled. To solve the problem, we can do one of two things. We can shift the result back to the right by 16 bits after the multiplication, or shift one of the multiplicands to the right by 16 bits before the multiplication. This will result in an answer that is scaled correctly. So, we could do this

```
f3 = (f1*f2)>>16;
```

or this:

```
f3 = ((f1>>16) * f2);
```

However, now we have a problem. Take a look at Figure 10-36. If we shift the results after the multiplication, it's possible that we may get an overflow since, in the worst case, two 32-bit numbers can result in a 64-bit product (actually, in this particular 16.16 representation, whenever numbers greater than 1.0 are multiplied there will be an overflow). On the other hand, if we shift one of the numbers to the right before the multiplication, we diminish our accuracy since we are in essence throwing away the decimal portion of one of the numbers. This can be remedied a bit by shifting each number by 8 bits and symmetrically decreasing their accuracy. However, the best solution to the problem is to use 64-bit math.

Using 64-bit math is possible, but it isn't easy to do with a 16-bit C/C++ compiler. Even with the in-line assembler that most C/C++ compilers come with, performing 64-bit math may not be possible if the compiler doesn't support 32-bit instructions. But we will deal with all these problems later. If worse comes to worst, we can always link in external assembly language functions that perform full 64-bit integer fixed point mathematics. Don't miss the little hidden lesson here, which is this: fixed point math performed with 32-bit numbers on a 16-bit compiler is slower than floating point on 486DXs and Pentiums! This is because the compiler turns simple operations like

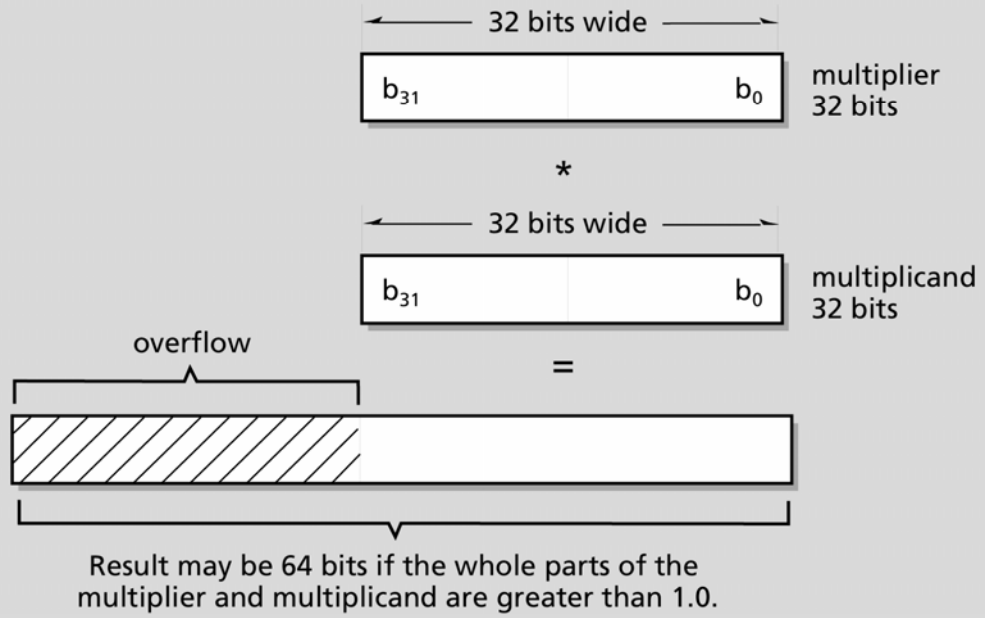

FIGURE 10-36 ◎ *Fixed point numbers overflowing during multiplication*

```
long a,b;
a=a+b;
```

into multiple 16-bit operations, killing the speed gained from the whole fixed point format.

Finally, division is performed much the same way as multiplication, but the numbers after division are underscaled! This occurs since there is a factor of 65536 in each dividend and divisor, and this factor will be divided out during the division. Thus, before the division is performed, the dividend must be scaled by another 16 bits–that is shifted to the left, for example

```
f3 = ((f1 <<16) / f2);
```

Once again this poses a problem, since during the shift to the left of the dividend, the 32-bit fixed point number may overflow, as shown in Figure 10-37. We can come up with a "best of the worst" solution by scaling each number up 8 bits and then hoping for the best, but the best way to solve the problem is to use 64-bit math again. Therefore, our final library, to be totally accurate, must use 64-bit fixed point math to hold the overflows and underflows. However, if all the math we per-

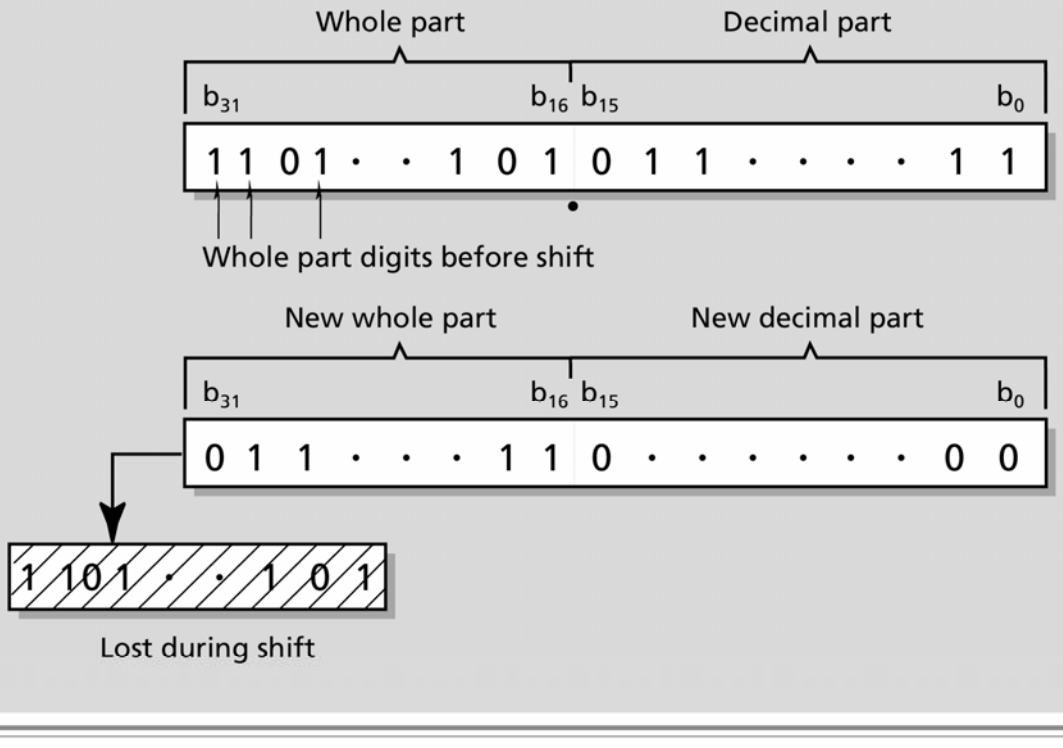

FIGURE 10-37 ◉ *Losing accuracy before division*

form can't possibly overflow or underflow in 32 bits, we can get away with 32-bit math. These are decisions that we will make as we design the 3D graphics engine.

Finally, to convert a fixed point number back to an integer, all we need to do is shift it back to the right 16 bits, as in:

```
int number;

number = (f1>>16);
```

Making the Quantum Leap to 3D

Now that we have laid the mathematical foundation necessary to discuss 3D graphics, let's begin by realizing that 3D graphics is not hard, not beyond comprehension. However, there are many details that we have to consider. As long as we take it slow and understand all the material and concepts, we'll have no trouble at all. The remaining material in this chapter is going to be an overall coverage of 3D graphics. Rather than describing any single topic in detail, we are going to see how it all fits together.